

5.8.27

Jnanesh Sathisha karmar - EE25BTECH11029

October 12,2025

Question:

$\mathbf{A} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & -2 & 4 \end{pmatrix}$ and $\mathbf{I} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$ and $\mathbf{A}^{-1} = \frac{1}{6}(\mathbf{A}^2 + c\mathbf{A} + d\mathbf{I})$, then value of c and d are

① $(-6, -11)$

② $(6, 11)$

③ $(-6, 11)$

④ $(6, -11)$

Theoretical Solution

Given the matrices

$$A = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & -2 & 4 \end{pmatrix} \quad \text{and} \quad I = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \quad (1)$$

and the relation $\mathbf{A}^{-1} = \frac{1}{6}(\mathbf{A}^2 + c\mathbf{A} + d\mathbf{I})$

First we Isolate the Unknowns

Starting with the given relation:

$$\mathbf{A}^{-1} = \frac{1}{6}(\mathbf{A}^2 + c\mathbf{A} + d\mathbf{I}) \quad (2)$$

Multiply both sides by 6 and rearrange to isolate the terms with the unknown scalars.

$$6\mathbf{A}^{-1} = \mathbf{A}^2 + c\mathbf{A} + d\mathbf{I} \quad (3)$$

$$c\mathbf{A} + d\mathbf{I} = 6\mathbf{A}^{-1} - \mathbf{A}^2 \quad (4)$$

Theoretical Solution

We find the inverse using Gauss-Jordan elimination on the augmented matrix $(\mathbf{A} \mid \mathbf{I})$.

$$\left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & -2 & 4 & 0 & 0 & 1 \end{array} \right) \xrightarrow{R_3 \rightarrow R_3 + 2R_2} \left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 6 & 0 & 2 & 1 \end{array} \right) \xrightarrow{R_2 \rightarrow R_2 - \frac{1}{6}R_3} \left(\begin{array}{ccc|ccc} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 5/6 & -1/6 \\ 0 & 0 & 6 & 0 & 2 & 1 \end{array} \right) \quad (5)$$

After scaling the last row, we get $(\mathbf{A} \mid \mathbf{I})$. The resulting inverse matrix is:

$$\mathbf{A}^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2/3 & -1/6 \\ 0 & 1/3 & 1/6 \end{pmatrix} \quad (6)$$

Now we Calculate the Square Matrix, $\mathbf{A}^2$ We compute $\mathbf{A}^2$ by matrix multiplication.

Theoretical Solution

$$\mathbf{A}^2 = \mathbf{A} \cdot \mathbf{A} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & -2 & 4 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & -2 & 4 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 5 \\ 0 & -10 & 14 \end{pmatrix} \quad (7)$$

Substituting the calculated matrices into Equation (2):

$$\begin{aligned} 6\mathbf{A}^{-1} - \mathbf{A}^2 &= 6 \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2/3 & -1/6 \\ 0 & 1/3 & 1/6 \end{pmatrix} - \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 5 \\ 0 & -10 & 14 \end{pmatrix} \\ &= \begin{pmatrix} 6 & 0 & 0 \\ 0 & 4 & -1 \\ 0 & 2 & 1 \end{pmatrix} - \begin{pmatrix} 1 & 0 & 0 \\ 0 & -1 & 5 \\ 0 & -10 & 14 \end{pmatrix} \\ &= \begin{pmatrix} 5 & 0 & 0 \\ 0 & 5 & -6 \\ 0 & 12 & -13 \end{pmatrix} \end{aligned}$$

Theoretical Solution

Now, set the left side of Equation (2) equal to this result:

$$c \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & -2 & 4 \end{pmatrix} + d \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 5 & 0 & 0 \\ 0 & 5 & -6 \\ 0 & 12 & -13 \end{pmatrix} \quad (8)$$

$$\begin{pmatrix} c+d & 0 & 0 \\ 0 & c+d & c \\ 0 & -2c & 4c+d \end{pmatrix} = \begin{pmatrix} 5 & 0 & 0 \\ 0 & 5 & -6 \\ 0 & 12 & -13 \end{pmatrix} \quad (9)$$

By equating corresponding elements, we can find the values of c and d .

From element (2, 3): $c = -6$

From element (2, 2): $c + d = 5 \implies -6 + d = 5 \implies d = 11$

The solution is therefore third option $(c, d) = (-6, 11)$.

C Code

```
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <math.h>

#define N 3

void printMatrix(double mat[N][N], const char* name);
void multiplyMatrices(double mat1[N][N], double mat2[N][N],
    double result[N][N]);
bool invertMatrix(double mat[N][N], double inverse[N][N]);
void scalarMultiply(double scalar, double mat[N][N], double
    result[N][N]);
void subtractMatrices(double mat1[N][N], double mat2[N][N],
    double result[N][N]);

int main() {
    double A[N][N] = {
        {1.0, 0.0, 0.0},
```

```
printf("--- Initial Matrix A ---\n");
printMatrix(A, "A");
multiplyMatrices(A, A, A_squared);
printf("\n--- Step 1: Calculated A^2 ---\n");
printMatrix(A_squared, "A^2");
printf("\n--- Step 2: Calculating A^-1 ---\n");
if (!invertMatrix(A, A_inverse)) {
    printf("Matrix A is not invertible. Cannot solve.\n");
    return 1;}
printMatrix(A_inverse, "A^-1");
printf("\n--- Step 3: Calculating RHS = 6*A^-1 - A^2 ---\n");
scalarMultiply(6.0, A_inverse, temp_matrix);
printMatrix(temp_matrix, "6 * A^-1");
subtractMatrices(temp_matrix, A_squared, RHS_matrix);
printMatrix(RHS_matrix, "RHS Matrix");
double c = RHS_matrix[1][2];
double d = RHS_matrix[1][1] - c;
```



```

    printf("\n--- Step 4: Solved for c and d ---\n");
    printf("From element (2, 3) of the matrix equation, we get c
        = %.2f\n", c);
    printf("From element (2, 2), we get c + d = %.2f, so d = %.2f
        - %.2f = %.2f\n", RHS_matrix[1][1], RHS_matrix[1][1], c,
        d);
    printf("\n===== \n");
    printf(" Final Solution \n");
    printf("===== \n");
    printf("c = %.2f\n", c);
    printf("d = %.2f\n", d);
    printf("===== \n");
    return 0;}

void printMatrix(double mat[N][N], const char* name) {
    printf("Matrix %s:\n", name);
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {printf("%8.3f ", mat[i][j])
            ;}
        printf("\n");
    }
}

```

C Code

```
void multiplyMatrices(double mat1[N][N], double mat2[N][N],
    double result[N][N]) {
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            result[i][j] = 0;
            for (int k = 0; k < N; k++) {
                result[i][j] += mat1[i][k] * mat2[k][j];
            }
        }
    }
}

bool invertMatrix(double mat[N][N], double inverse[N][N]) {
    double augvec[N][N * 2];

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            augvec[i][j] = mat[i][j];
        }
    }
}
```

```
for (int i = 0; i < N; i++) {
    double pivot = augvec[i][i];

    if (pivot == 0) {
        int max_row = i;
        for (int k = i + 1; k < N; k++) {
            if (fabs(augvec[k][i]) > fabs(augvec[max_row][i]))
                {
                    max_row = k;
                }
        }

        if (max_row != i) {
            for (int j = 0; j < 2 * N; j++) {
                double temp = augvec[i][j];
                augvec[i][j] = augvec[max_row][j];
                augvec[max_row][j] = temp;
            }
        }
    }
}
```

C Code

```
    pivot = augvec[i][i];
    if (pivot == 0) return false;
    for (int j = 0; j < 2 * N; j++) {augvec[i][j] /= pivot;}
    for (int k = 0; k < N; k++) {
        if (k != i) {
            double factor = augvec[k][i];
            for (int j = 0; j < 2 * N; j++) {
                augvec[k][j] -= factor * augvec[i][j];}
        }
    }
}
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        inverse[i][j] = augvec[i][j + N];
    }
}
return true;
}
```

```
void scalarMultiply(double scalar, double mat[N][N], double
    result[N][N]) {
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        result[i][j] = mat[i][j] * scalar;
    }
}

}

void subtractMatrices(double mat1[N][N], double mat2[N][N],
    double result[N][N]) {
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        result[i][j] = mat1[i][j] - mat2[i][j];
    }
}

}
```